圆形检测

圆形检测
- 例程实验效果简介
- 代码讲解
  - 完整代码
  - 代码结构
- 圆形检测函数
  - `find_rects`

例程实验效果简介

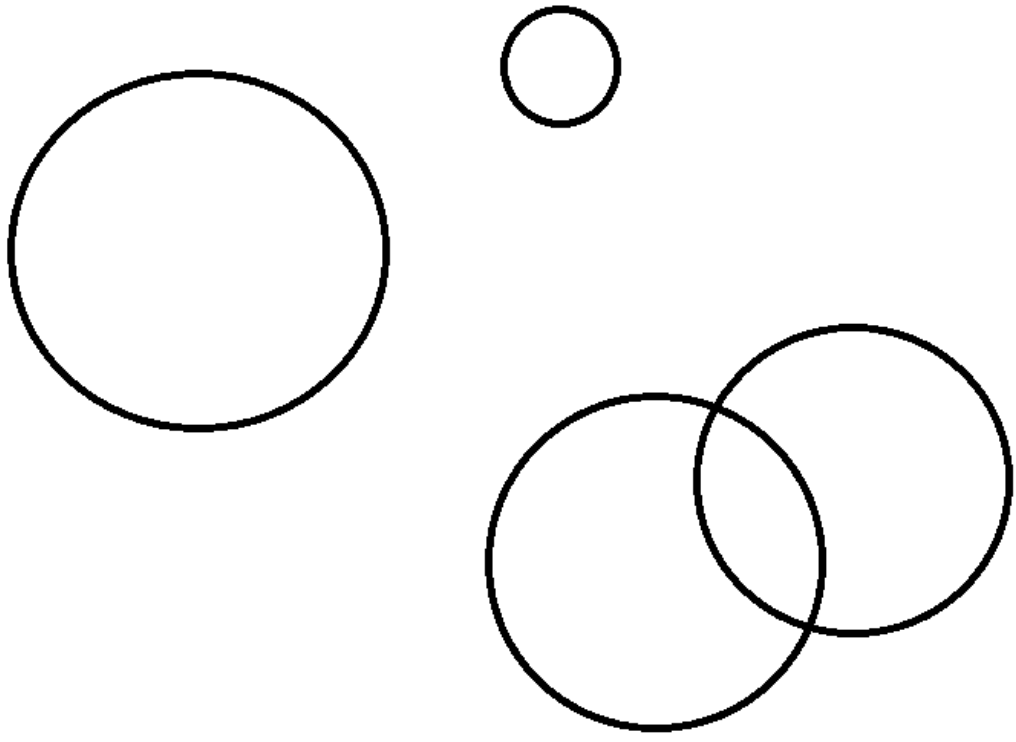
本节例程代码位于：【源码汇总 / 04.Detecting / 03.find_circles.py】

我们用CanMV IDE打开例程代码，将K230用usb连接到电脑上

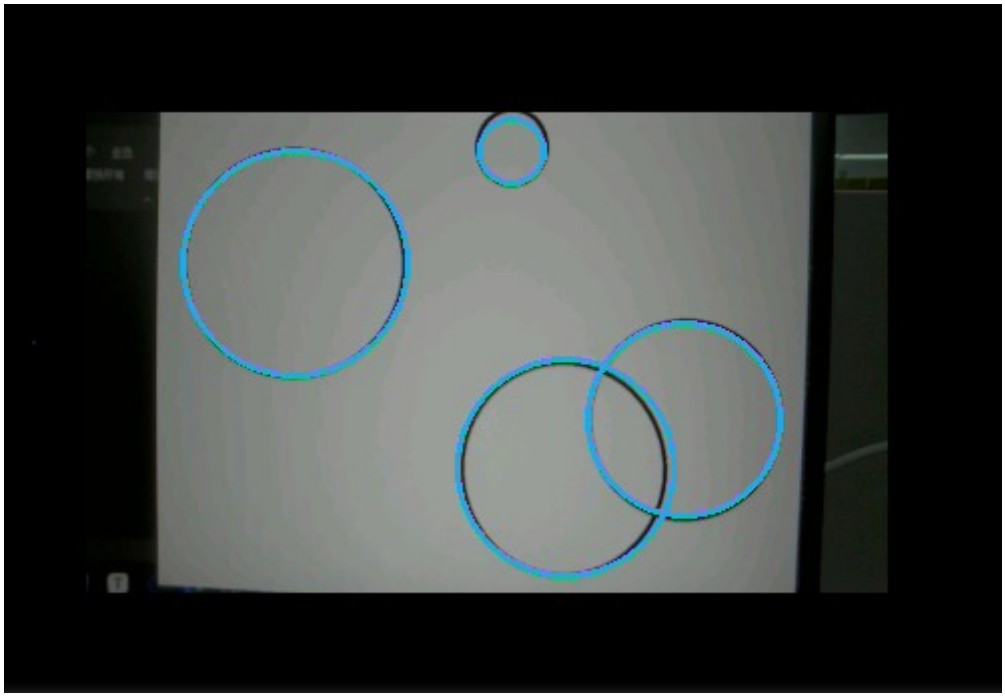
点击CanMV IDE左下角的运行按钮，

将K230的摄像头对准线段

可以看到屏幕上会标记出画面中的**圆形**（如果没有屏幕就看帧缓冲区）



【K230识别结果】



代码讲解

本节我们要用的的外设主要是摄像头模块

线段检测由 K230中的 find_circles() 方法实现，该方法属于image模块

完整代码

```
import time, os, sys
from media.sensor import *
from media.display import *
from media.media import *

# 设置图像捕获分辨率 / Set picture capture resolution
PICTURE_WIDTH = 400
PICTURE_HEIGHT = 240

# 显示模式：“VIRT”用于无屏幕情况，“LCD”用于有屏幕情况
# Display mode: "VIRT" for virtual display, "LCD" for physical screen
DISPLAY_MODE = "LCD"

# 根据显示模式设置显示分辨率 / Set display resolution based on mode
if DISPLAY_MODE == "VIRT":
    # 虚拟显示器模式(1920x1080) / Virtual display mode
    DISPLAY_WIDTH = ALIGN_UP(1920, 16)  # 确保宽度16字节对齐 / Ensure width is 16-
byte aligned
    DISPLAY_HEIGHT = 1080
elif DISPLAY_MODE == "LCD":
    DISPLAY_WIDTH = 640
    DISPLAY_HEIGHT = 480
else:
    raise ValueError("未知的显示模式，请选择 'VIRT' 或 'LCD' / Unknown display mode,
please choose 'VIRT' or 'LCD'")
```

```python
def init_sensor():
    """初始化传感器 / Initialize sensor"""
    sensor = Sensor(id=2)
    sensor.reset()
    sensor.set_framesize(width=PICTURE_WIDTH, height=PICTURE_HEIGHT,
chn=CAM_CHN_ID_0)
    sensor.set_pixformat(Sensor.RGB565, chn=CAM_CHN_ID_0)
    return sensor

def init_display():
    """初始化显示器 / Initialize display"""
    if DISPLAY_MODE == "VIRT":
        Display.init(Display.VIRT, width=DISPLAY_WIDTH, height=DISPLAY_HEIGHT,
fps=60)
    elif DISPLAY_MODE == "LCD":
        Display.init(Display.ST7701, width=DISPLAY_WIDTH, height=DISPLAY_HEIGHT,
to_ide=True)

def process_circles(img, circles):
    """处理检测到的圆形 / Process detected circles"""
    print("【圆形信息 / Circle Statistics Start】")
    for i, circle in enumerate(circles):
        # 使用蓝色绘制圆形 / Draw circles in blue
        img.draw_circle(circle.circle(), color=(40,167,225), thickness=3)
        print(f"Circle {i}: {circle}")
    print("【============================】")

def main():
    try:
        # 初始化设备 / Initialize devices
        sensor = init_sensor()
        init_display()
        MediaManager.init()
        sensor.run()

        # 计算显示偏移量以居中显示 / Calculate display offsets for center alignment
        x_offset = (DISPLAY_WIDTH - PICTURE_WIDTH) // 2
        y_offset = (DISPLAY_HEIGHT - PICTURE_HEIGHT) // 2

        while True:
            os.exitpoint()

            # 捕获图像 / Capture image
            img = sensor.snapshot(chn=CAM_CHN_ID_0)

            # 寻找并处理圆形 / Find and process circles
            # threshold: 控制圆形检测的阈值，更高的值会减少检测到的圆形数量
            # threshold: Controls circle detection sensitivity, higher values
reduce the number of detected circles
            circles = img.find_circles(threshold=3500)
            process_circles(img, circles)

            # 居中显示图像 / Display image in center
            Display.show_image(img, x=x_offset, y=y_offset)

    except KeyboardInterrupt as e:
        print("用户中断 / User interrupted: ", e)
    except Exception as e:
```

```
        print(f"发生错误 / Error occurred: {e}")
    finally:
        # 清理资源 / Cleanup resources
        if 'sensor' in locals() and isinstance(sensor, Sensor):
            sensor.stop()
        Display.deinit()
        os.exitpoint(os.EXITPOINT_ENABLE_SLEEP)
        time.sleep_ms(100)
        MediaManager.deinit()

if __name__ == "__main__":
    main()
```

代码结构

初始化配置

- 支持两种显示模式：虚拟显示器(VIRT, 1920x1080)和LCD屏幕(640x480)
- 根据选择的显示模式设置相应的显示分辨率

核心功能函数

- init_sensor(): 初始化摄像头传感器，设置图像分辨率和格式
- init_display(): 根据显示模式初始化显示设备
- process_circles(): 处理检测到的圆形，在图像上绘制并输出信息

主循环处理

- 持续捕获图像
- 在图像中检测圆形
- 在检测到的圆形位置绘制蓝色圆圈
- 将处理后的图像居中显示在屏幕上

异常处理和资源清理

- 处理键盘中断等异常情况
- 程序结束时确保正确释放所有资源（传感器、显示器、媒体管理器）

圆形检测函数

`find_rects`

```
image.find_rects([roi=Auto, threshold=10000])
```

此函数使用与 AprilTag 相同的四边形检测算法查找图像中的矩形。该算法最适用于与背景形成鲜明对比的矩形。AprilTag 的四边形检测能够处理任意缩放、旋转和剪切的矩形，并返回一个包含 `image.rect` 对象的列表。

- `roi` 是用于指定感兴趣区域的矩形元组 `(x, y, w, h)`。若未指定，ROI 默认为整个图像。操作范围仅限于该区域内的像素。

在返回的矩形列表中，边界大小（通过在矩形边缘的所有像素上滑动索贝尔算子并累加其值）小于 `threshold` 的矩形将被过滤。适当的 `threshold` 值取决于具体的应用场景。

注意： 不支持压缩图像和 Bayer 图像。